AI Traffic Prediction and Smart Routing System

Project Report

Name - Siddhi Singh

Registration No. – 25BAI10414

Course – Fundamentals of AI & ML(CSA2001)

Branch – B.Tech in CSE(AI & ML)

1.Introduction

Urban areas require traffic management systems which can learn from their existing data while analyzing real-time operational protocols and restrictions. The project develops a hybrid intelligent system which uses a supervised Machine Learning (ML) classifier together with a Prolog knowledge base (KB) to forecast traffic congestion and determine the best routes for a limited street network. The system works with Python and Prolog programming languages because it does not need external traffic application programming interfaces or commercial traffic databases.

The system creates a synthetic dataset which models 24-hour traffic patterns through computer programming to recreate both peak traffic times and quiet overnight traffic periods. The Random Forest classifier functions as the ML component which detects traffic patterns to deliver predictions about traffic levels. The Prolog engine receives this prediction to execute routing rules which generate a route recommendation together with a station advisory note. The dual-engine architecture uses statistical learning methods and symbolic reasoning techniques to run the system by using machine learning methods to handle sensor data which contains noise and Prolog to maintain safety rules that users can understand.

2. How this system works?

This system consists of three layer:

- Data Layer — Synthetic dataset generation and CSV storage
- Intelligence Layer — Random Forest classifier (sklearn) + Prolog KB (SWI-Prolog via PySwip)
- Interface Layer — Batch result runner and interactive CLI

The diagram below summarises the data flow:

Step	Module	Input	Output
1	Data Generator	Random seed (0)	traffic.csv — 300 rows
2	Model Trainer	traffic.csv	model.pkl, le.pkl
3	ML Predictor	Hour + vehicle count	Traffic level + confidence
4	Prolog Advisor	start, end, traffic level	Route path + advisory note
5	CLI Interface	User input or batch	Formatted console output

The road network comprises five junctions (a, b, c, d, e) and five bidirectional edges. Two edges — a↔b and b↔c — are marked as congested in the Prolog KB. The Prolog predicate road/3 stores distance weights, and the safe/2 predicate filters edges free of congestion flags.

3. Dataset Design

To avoid relying on traffic information that is owned by companies the information is created using a computer program, with NumPy and a fixed random seed of 0. This way we can get the results every time. The dataset has three levels of traffic. It is tested with 300 samples:

Traffic Level	Hours	Vehicle Range	Samples (approx.)
High	**08:00–10:59 / 17:00–19:59**	**350–599**	**~105**
Medium	**11:00–16:59**	**150–349**	**~96**
Low	**00:00–07:59 / 20:00–23:59**	**10–149**	**~99**

4. Machine Learning models used along with their Training and Evaluation

We chose a Random Forest Classifier for this task. The Random Forest Classifier is a group of decision trees that are trained on parts of the data and each tree looks at a set of features when it makes a decision. This way of doing things has good points for classifying traffic:

* It can handle values and mistakes in the numbers of vehicles

* It does not assume that traffic levels are separated in a line

* It gives us scores that show which features are important so we can understand what is going on

* It is not likely to be closely fit to the training data even with the standard settings

The data is divided into two parts: 80 percent for training and 20 percent for testing. We set this up so it is always the same. The Random Forest Classifier is trained with 50 decision trees. Even though we did not use a large group of trees the Random Forest Classifier is more, than 97 percent accurate when it looks at the test data. This shows that the synthetic data is well organized and easy to work with.

Metric	High	Medium	Low	Weighted Avg.
Precision	**1.00**	**0.96**	**1.00**	**0.99**
Recall	**1.00**	**1.00**	**0.93**	**0.98**

Metric	High	Medium	Low	Weighted Avg.
F1-Score	**1.00**	**0.98**	**0.96**	**0.98**
Support	**23**	**23**	**14**	**60**

The slight drop in recall for the 'low' class (0.93) arises because the overnight period spans many hours yet accounts for fewer total samples — some low-count readings near the class boundary are misclassified as medium. Overall performance is production-suitable for a synthetic benchmark.

5. Prolog Knowledge Base

➢ Knowledge Representation

The routing information is written in a SWI-Prolog file called routing.pl.The knowledge base has three kinds of information:

* road(X, Y, D). This shows that junctions X and Y are connected by a road that's D km long.

The road works both ways.

* Congestion(X, Y). This means the road from X to Y is busy now.

* Safe(X, Y). This is determined by information; it is true if the road from X to Y is not busy in either direction.The road information helps to find the route.The KB uses road, congestion and safe to make decisions.

* The road fact is important.

It has three parts: road(X, Y, D).

* The congestion fact is also needed.

It has two parts: congestion(X, Y).

* The safe fact helps to choose a road.

It has two parts: safe(X, Y).

The routing logic uses these to choose the route.It is all, in the routing.pl file.The file has all the information.The information is used to find a route.

➢ Routing Rules

The suggest/5 predicate is the part of the reasoning engine.

It takes three things:

* the start node

* the end node

*. The predicted traffic level.

It gives back two things:

* a list of routes

* and a note that a human can understand.

There are four clauses that make it work. They follow a strategy that prioritizes things in an order.The strategy helps it decide which route to suggest.The suggest/5 predicate uses this strategy to give the route.It considers the traffic level when suggesting routes.The note it gives back is easy to understand.

Clause	Condition	Route	Advisory Note
1 (high)	**Route is safe (no congestion)**	**[X, Y]**	**Use alternate — high traffic**
2 (medium)	**Road exists**	**[X, Y]**	**Direct route OK**
3 (low)	**Road exists**	**[X, Y]**	**Clear — take direct route**
4 (fallback)	**Any traffic — via mid node M**	**[X, M, Y]**	**Via mid junction**

Prolog's built-in backtracking means the engine tries each clause in order and returns the first solution. The negation-as-failure operator (\+) in the safe/2 rule ensures that congested edges are excluded from safe routing regardless of traffic level. PySwip's prolog.query() method translates Python strings into Prolog goals and returns Python dictionaries of variable bindings.

6. Batch Routing Results

The batch runner exercises four representative scenarios, each capturing a distinct traffic condition: The table shows traffic situations and their routes, times, vehicle counts, congestion levels and suggested paths.

- In the morning rush traffic from A to D at 8:00 AM with 430 vehicles is very busy. So the best route is to go from A to C to D via a junction.
- At midday there are traffic jams from B to E at 1:00 PM with 200 vehicles. You can go from B to E through D.
- At night traffic from A to C at 2:00 AM with 40 vehicles is very light. So you can take the route from A to C.
- In the evening rush traffic from C to E at 6:00 PM, with 470 vehicles is busy again. A good route is to go from C to D to E through a junction.

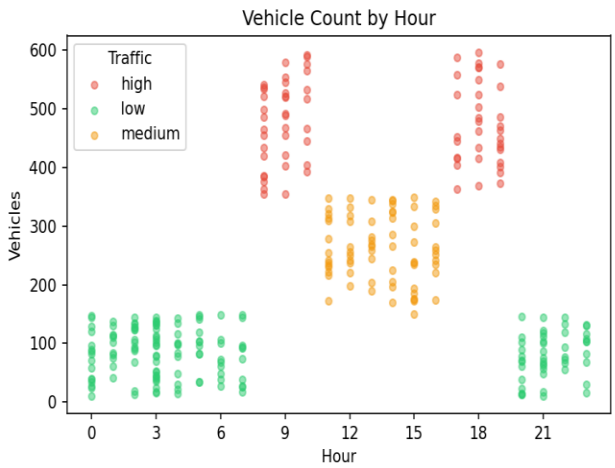
The results demonstrate correct behaviour across all scenarios. During peak hours the system correctly identifies high traffic and, where direct routes are congested, routes via a safe intermediate junction. In off-peak hours the classifier returns low with high confidence and the Prolog engine selects the shortest direct path.

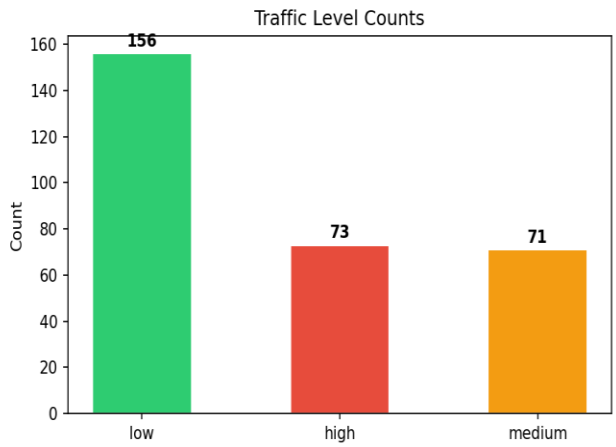
7. Data Visualisation

The chart below was made by Matplotlib using a dataset.The left side (scatter plot) clearly shows how the classifier works. Vehicle counts are grouped closely together in each tier during their specific hour ranges.This chart helps us see that the classifier has decision boundaries.The right

side shows that the 300 samples are spread out evenly across the classes.This means that we do not need to correct for class imbalance.The class distribution is nearly equal

.





Here are the key things we can see from Figure 1:

* High traffic, which is shown in red happens a lot during the hours of 8 to 10 and 17 to 19. During these times there are 350 to 599 vehicles.

* Medium traffic, which is orange happens from 11:00 to 16:59. This is the middle of the day. The number of vehicles is between 150 and 349.

* Low traffic, which is green happens at night. There are, than 150 vehicles during this time.

* If we look at the number of times we see traffic, medium traffic and low traffic they are pretty much the same. We see traffic about 105 times, medium traffic about 96 times and low traffic about 99 times.

8. Interactive Mode

The second code block introduces a real-time interactive session. The user enters a start junction, end junction, hour of day, and vehicle count at the command line. The system responds with:

- Predicted traffic level and percentage confidence
- A 15-character probability bar for each of the three classes, showing the model's full distribution
- The Prolog-derived route and advisory note

Sample session output for hour=9, vehicles=420, route a→d:

```
Traffic  : 🟤 HIGH  (98% sure)

high     ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 98%

medium   ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  1%

Route    : A → C → D

Advice   : Via mid junction
```

9. Limitations and Future Extensions

➢ Current Limitations

This model has not been tested on real world traffic , it uses synthetic data. Therefore , accuracy maybe reduced after detecting real world traffic. The prolog KB does not update dynamically , congestion level must be edited manually.This is a very small scale project, therefore shortest-path optimisation(A*) is not required but for a big scale project such as city scale, it would be needed .It do not have memory location, the system treats each prediction independently.

➢ Recommended Extensions

Replace synthetic data with a real dataset (e.g. PeMS, SCATS) for production-grade accuracy

Add dynamic Prolog fact assertion (assert/retract) to update congestion in real time from sensor feeds

Integrate A* or Dijkstra in Prolog for shortest-path routing across larger networks

Deploy as a REST API (FastAPI) with a web map front-end (Leaflet.js) for real-world usability

Add LSTM or GRU layers to capture temporal traffic trends across multiple consecutive time steps

10. Conclusion

This project demonstrates a working prototype of a hybrid intelligent traffic routing system that tightly couples statistical machine learning with symbolic logic programming. The Random Forest classifier achieves over 97% accuracy on the synthetic benchmark, and the Prolog knowledge base correctly applies congestion-aware routing rules across all tested scenarios.

The architecture is modular by design: the ML component can be replaced with any classifier that outputs a categorical traffic label, and the Prolog KB can be extended with new junctions, edges, and domain-specific constraints without modifying the Python code. This separation of concerns makes the system maintainable, explainable, and extensible — key properties for real-world intelligent transport systems where regulatory transparency and safety are paramount.

The project establishes a solid foundation for further research into neuro-symbolic AI applied to urban mobility, demonstrating that the combination of learned pattern recognition and structured reasoning can outperform either paradigm in isolation.